A Learning-to-Optimize approach for the Random Walk Interdiction Problem

Abstract: We introduce the *Random Walk Interdiction* problem, a stochastic variant of the classical shortest-path interdiction framework in which an attacker follows a Markovian random walk toward a target node and the defender decides which nodes in the graph it should protect, and therefore stop the attacker. We first design a message-passing algorithm to compute a provable approximate solution of the problem and then we construct a random-basis representation that enables approximated solution learning through an efficient stochastic gradient ascent. Numerical experiments on synthetic directed acyclic graphs confirm that the proposed learning scheme rapidly converges to near-optimal interdiction strategies while remaining computationally scalable.

Keywords: Random Walk Interdiction Problem, Message Passing Algorithm, Learning to Optimize

1. INTRODUCTION

Cyberattacks are becoming more and more frequent and increasingly sophisticated, making the automation of defense strategies a priority. One strategy to address the challenges behind automation in such context is by considering an optimisation-centric approach. Previous works in cybersecurity have studied the shortest path edge interdiction problem in attack graphs Guo et al. (2022); Ngo et al. (2024), and the well known Stackelberg Security game Sinha et al. (2018). Despite extensive research in this area, current approaches face one major limitation. These problems remain computationally intensive due to the inherent non-convexity of the problem, requiring solving the optimisation problem from scratch for each new instance, limiting their practicality for real-time or large-scale applications.

In this paper, we focus on a new optimisation problem called the *Random Walk Interdiction problem*, a stochastic variant of the classical shortest-path interdiction. In this problem, we model the attacker as a random walker on a graph trying to reach a designated target node. Such dynamic models are commonly used to study malware spread or lateral movement attacks. The objective of the defender is to minimize the hitting probabilities of the target by removing a limited number of nodes from the graph. A natural approach to solve this problem is to study a relaxed version, where each node is removed with a certain probability. This leads to a non-convex optimisation problem like in Borkar and Reiffers-Masson (2021).

To develop learning-based solver for the problem defined above, we adopt in this paper the learning-to-optimize (L2O) paradigm Chen et al. (2022), where neural networks are trained to mimic classical optimisation algorithms. From the best of our knowledge, no one has designed function approximation or Neural Networks that capture optimal approximation algorithms for this type of problem. Usually Graph Neural Networks (GNN) are used in L2O approach for optimisation problems with graphs. GNNs have already demonstrated success across several optimisation tasks, including: 1) Linear programming, where GNNs can simulate interior-point methods Qian et al. (2024); Chen et al. (2023); 2) Max Constraint Satisfaction Problems Yau et al. (2024), where polynomial-sized message-passing networks match known approximation algorithms; 3) Classical combinatorial problems such as Vertex Cover and Dominating Set Sato et al. (2019). Recent papers on resource allocation Reiffers-Masson et al. (2021) and opinion control Borkar and Reiffers-Masson (2021) have been able to design message passing algorithm (deterministic or stochastic) to solve their respective optimisation problem. However, the problems studied are convex, and do not focus on how to neuralize the obtained algorithms.

Outline and Contributions: In section 2 we define the Random Walk Interdiction problem for which we are trying to learn the solver and we state our main approximation result. In Section 3, we first show that there exists a message-passing algorithm that can be used to approximate the gradient. We also prove, in Theorem 3.4, that such message-passing algorithm, when coupled with a classical gradient ascent algorithm, achieves a provable approximation of the optimal solution. Building on this result, we then show in Section 4 how to construct a random basis representation with theoretical approximation guarantees (see Proposition 4.2). In Section 5, we derive an efficient algorithm to train our random basis model based on the efficient Monte-Carlo scheme on the score-function gradient. Finally, in Section 6, we evaluate the performance of the proposed random basis approach and compare the efficiency of our learning algorithm with standard gradient-based schemes on synthetic data.

2. MODEL FORMULATION

We consider a directed graph $G = (V, E)$ where V is the set of nodes and E the set of edges. We assume that the behavior of the attacker on the graph G is captured a Markovian process on G. The transition probability matrix is given by $P \in \mathbb{R}^{|V|\times|V|}$, where P_{ij} denotes the probability of the process to go from node i to node j in one step. For each node $i \in V$, we define the *hitting probability* h_i as the probability that the attacker starting from

node i eventually reaches the target node T. The hitting probabilities satisfy the following recursive equation:

$$h_i = (1-\pi_i)\sum_{j\in V} P_{ij}h_j, \quad \forall i \neq T, \tag{1}$$

where $\pi_i \in [0,1]$ is the probability of removing node i. The boundary condition is given by $h_T = 1$.

Our objective is to maximize the weighted sum of the hitting probabilities across all nodes while satisfying a budget constraint on the total removal probability. More precisely, the Random Walk Interdiction problem is formulated as follows:

$$\max_{\pi} \quad J(\pi) = \sum_{i\in V} w_i(1-h_i) \tag{2}$$

$$\text{s.t. } h_i - (1-\pi_i)\sum_{j\in V} P_{ij}h_j = 0,\ \forall i \neq T,\ h_T = 1, \tag{3}$$

$$\sum_{i\in V}\pi_i \leq \phi, \tag{4}$$

$$0 \leq \pi_i \leq 1, \quad \forall i \in V, \tag{5}$$

where, w_i represents the importance weight of node i and ϕ is the total budget for node removal probabilities. Observe that constraints (3) ensure that the hitting probabilities satisfy the recursive equation, the constraint (4) enforces the global budget on node removals and the constraint (5) ensures that removal probabilities remain valid probabilities.

Our main result is the following:

Theorem 2.1 (Informal Statement)**.** *There exists a message-passing algorithm—based on fixed-point updates of the hitting probabilities and dual variables—such that, when initialized appropriately and run for n iterations, it produces a solution π^n satisfying:*

$$J(\pi^n) \geq \frac{1}{2}\max_{\pi\in\mathcal{C}} J(\pi) - K(\epsilon, n).$$

where

- $\mathcal{C} = \left\{\pi \in [0,1]^{|V|} \,\middle|\, \sum_i \pi_i \leq \phi\right\}$,
- *ϵ quantifies the gradient approximation error due to finite fixed-point iterations,*
- *n is the number of gradient updates,*
- *and* $\lim_{n\to+\infty,\epsilon\to 0} K(\epsilon, n) = 0$.

Proof sketch: Our proof is structured in three main steps. In the first step, we show that first order optimality condition can be expressed as a linear implicit Graph Neural Network (GNN)—i.e., a message-passing architecture that solves a linear fixed-point equation. In the second step, we prove that the underlying optimization problem can be cast as a submodular maximization problem. This allows us to invoke known results (theorem 4.2 of Hassani et al. (2017)) which guarantee that the stationnary points of $J(\pi)$, denoted by π^* are at least $\frac{1}{2}$ approximation in the sense that:

$$\max_{\pi} J(\pi) \geq J(\pi^*) \geq \frac{1}{2}\max_{\pi} J(\pi).$$

Finally, we show that our proposed algorithm performs an approximate gradient ascent, where the gradient is not computed exactly but rather approximated via message-passing. We then provide bounds—under suitable initialization—on how far the resulting solution is from the true optimum, quantifying the impact of gradient approximation on convergence.

3. PROOF AND FORMAL STATEMENT OF THE THEOREM

3.1 Step 1: Message passing for gradient computation

Let us first consider the general bi-level optimization problem:

$$\max_{x} U(y) \tag{6}$$

$$\text{s.t. } y \text{ sol. of } y = g(x,y), \tag{7}$$

$$Ax = b,\ x \geq 0. \tag{8}$$

Our next result shows that treating y also as an explicit optimization variable yields first-order optimality conditions (Karush–Kuhn–Tucker or KKT conditions) that are equivalent to those obtained by considering only x, with y implicitly defined through a fixed-point relation $y = g(x,y)$.

Lemma 3.1. *If we assume that for all x, such that $Ax = b$ and $x \geq 0$, $(I - \nabla_y g(x, y(x)))$ is invertible, then the solutions of first order optimality conditions of the original bi-level problem* (6)-(8) *are the same as the solution of first order optimality conditions*

$$\max_{x,y} U(y) \tag{9}$$

$$s.t.\ y = g(x,y), \tag{10}$$

$$Ax = b,\ x \geq 0. \tag{11}$$

Proof. Let us write the Lagrangian for the general bi-level problem:

$$L(x,\gamma,\nu) = U(y(x)) + \gamma^T(Ax-b) + \nu^T x,$$

with $\nu \geq 0$. To compute the stationnary point of L in x, we need to use implicit differentiation. Since $y(x)$ satisfies $y = g(x,y)$, we differentiate both sides with respect to x:

$$\nabla_x y(x) = \nabla_x g(x, y(x)) + \nabla_x y(x)\nabla_y g(x, y(x)).$$

By rearranging, we get

$$\nabla_x y(x)(I - \nabla_y g(x, y(x))) \ = \nabla_x g(x, y(x))$$

Assuming $(I - \nabla_y g(x, y(x)))$ is invertible, we obtain:

$$\nabla_x y(x) = \nabla_x g(x, y(x))(I - \nabla_y g(x, y(x)))^{-1}.$$

Now, differentiating the objective function:

$$\nabla_x U(y(x)) = \nabla_x y(x)\nabla_y U(y(x))$$

Substituting $\nabla_x y(x)$:

$$\nabla_x U(y(x)) = \nabla_x g(x, y(x))(I - \nabla_y g(x, y(x)))^{-1}\nabla_y U(y(x)).$$

Now the stationnary point of L in x is given by:

$$\nabla_x U(y(x)) + A^T\gamma + \nu = 0.$$

Now let us consider the optimisation problem (9)- (11). Note that y is now also an optimisation variable of the problem. Define the Lagrangian:

$$\mathcal{L}(x,y,\lambda,\gamma,\nu) = U(y) + \lambda^T(g(x,y) - y) + \gamma^T(Ax-b) + \nu^T x.$$

Looking at the stationnary point in y we obtain:

$$\begin{aligned} &\nabla_y \mathcal{L} = 0, \\ &\Leftrightarrow \nabla_y U(y) + (\nabla_y g(x,y) - I)\lambda = 0, \\ &\Leftrightarrow \lambda = (I - \nabla_y g(x,y))^{-1}\nabla_y U(y). \end{aligned}$$

Doing the same in λ gives us $y = g(x, y)$. Finally when we derive the stationnary point of the Lagrangian in x we get:

$$\begin{aligned}
&\frac{\partial \mathcal{L}}{\partial x} = 0 \\
&\Leftrightarrow \nabla_x g(x,y)\lambda + A^T\gamma + \nu = 0 \\
&\Leftrightarrow \nabla_x g(x,y)(I - \nabla_y g(x,y))^{-1}\nabla_y U(y) + A^T\gamma + \nu = 0
\end{aligned}$$

Note that substituting λ, we obtain that the solutions of $\frac{\partial \mathcal{L}}{\partial x} = 0$ are the same as the one of $\frac{\partial \mathcal{L}}{\partial x} = 0$ for all γ and ν allowing us to conclude the proof. □

Now going back to our initial problem (2)-(5), lemma 3.1 tells us that we can optimize in π_i and h_i simultaneously. To solve this new optimization problem, we introduce Lagrange multipliers λ_i for the hitting probability constraints and μ for the budget constraint. The Lagrangian function is given by:

$$\begin{aligned}
\mathcal{L}(\pi, h, \lambda, \mu) = &\sum_{i \in V} w_i h_i \\
&+ \sum_{i \neq T} \lambda_i \left(h_i - (1 - \pi_i) \sum_{j \in V} P_{ij} h_j \right) \\
&+ \mu \left(\sum_{i \in V} \pi_i - \phi \right).
\end{aligned}$$

Taking the first-order derivatives, the KKT conditions are:

$$\frac{\partial \mathcal{L}}{\partial h_i} = w_i + \lambda_i - \sum_{j \neq T} \lambda_j (1 - \pi_j) P_{ji} = 0, \quad \forall i \neq T, \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial \pi_i} = \lambda_i \sum_{j \in V} P_{ij} h_j + \mu + \nu_i = 0, \quad \forall i, \tag{13}$$

$$h_i - (1 - \pi_i) \sum_{j \in V} P_{ij} h_j = 0, \quad \forall i \neq T, \tag{14}$$

$$\sum_{i \in V} \pi_i \leq \phi, \tag{15}$$

$$\mu(\sum_{i \in V} \pi_i - \phi) = 0, \quad \nu_i \pi_i = 0, \ \nu_i \geq 0 \quad \forall i. \tag{16}$$

3.2 Step 2: Submodularity

Now for the second step of the proof, we have to ask ourself how far from the optimum is a solution of (12)-(16). In our case we use the fact that, from a minimization perspective, the function that we optimize is a supermodular function.

Lemma 3.2. *For all i, h_i is a supermodular function.*

Proof. Let $\mathcal{P}_{i \to T}$ be the set of all walks of finite length from i to T. For a walk $p = (i_0 = i, i_1, \ldots, i_{k_p} = T)$ we define first the transition probability of the walk p to be $\prod_{\ell=0}^{k_p - 1} P_{i_\ell i_{\ell+1}}$ and the probability of not being stop by the defender $\prod_{\ell=0}^{k_p-1}(1 - \pi_{i_\ell})$. Then the hitting probability h_i can be written as:

$$h_i = \sum_{p \in \mathcal{P}_{i \to T}} \left(\prod_{\ell=0}^{k_p-1} (1 - \pi_{i_\ell}) \cdot \prod_{\ell=0}^{k_p-1} P_{i_\ell i_{\ell+1}} \right).$$

Note that for all i, j, k we have $\frac{\partial^2 h_i}{\partial \pi_j \partial \pi_k} \geq 0$ implying that all cross-second-order derivatives are positive implies that h_i is supermodular (see Bach (2019)).

□

3.3 Gradient Ascent

Now we need to design a message passing algorithm which tries to solve (12)-(16). A natural one is to use a gradient ascent algorithm:

$$\pi_i^{n+1} = \Gamma \left[\pi_i^n + \eta \lambda_i^n \sum_j P_{ij} h_j^n \right], \ \forall i \neq T \tag{17}$$

$$h_i^n = (1 - \pi_i^n) \sum_j P_{ij} h_j^n, \ \forall i \neq T, \ h_T^n = 1, \tag{18}$$

$$\lambda_i^n = -w_i + \sum_{j \neq T} \lambda_j^n (1 - \pi_j^n) P_{ji}, \ \forall i \neq T, \ \lambda_T^n = 0 \tag{19}$$

where $\Gamma[\cdot]$ denotes the projection onto the simplex of dimension $|V| - 1$ and η is the learning rate. Note that in the algorithm above the computation of the fixed points of (18) and (19) are not made explicit.

Note that the fixed point h^n and λ^n can be interpreted as message-passing operations Wu et al. (2022) over a graph. We assume that the simple fixed point iterations is enough to compute to solve the fixed points leading to the following algorithm:

Message Passing Algorithm for the Random Walk Interdiction Problem

Input: step-size parameter η, number of inner iterations K and $\pi_i(0)$ for all i.

For each iteration $n = 1, 2, \ldots$:

- *Initialize* $h_i^n(0)$, $\lambda_i^n(0)$;
- *For each iteration $k = 1, \ldots, K-1$ update:* If $i \neq T$:

$$h_i^n(k+1) = (1 - \pi_i^n) \sum_j P_{ij} h_j^n(k),$$

$$\lambda_i^n(k+1) = -w_i + \sum_{j \neq T} \lambda_j^n(k)(1 - \pi_j^n) P_{ji},$$

  Else: set $h_T^n(k+1) = 1$, $\lambda_i^n(k+1) = 0$.
- *Update:*

$$\pi_i^{n+1} = \Gamma \left[\pi_i^n + \eta \lambda_i^n(K) \sum_j P_{ij} h_j(K) \right].$$

The message passing algorithm is an approximate gradient ascent. Indeed the computation of the gradient is not exact. However we can derive convergence results using the following lemma:

Lemma 3.3. *Let us assume that it exists R such $\forall x_0 \in B(x^*, R)$, where x^* is a local maximum of a function $f(\cdot)$,*

such that $-f$ is C-strongly convex for all $x \in B(x^*, R)$. Then:

(1) if $f(\cdot)$ is L-smooth, and $\alpha \leq \frac{2}{L+C}$ the opertor $G(x) = \Gamma[x + \eta \nabla f(x)]$ is $\zeta := 1 - \frac{\eta CL}{C+L}$-expansive.

(2) If η is small enough, $G(x)$ is a contraction and we run approximated gradient ascent $x_{n+1} = \Gamma[x_n + \eta \hat{g}(x_n)]$, with $x_0 \in B(x^*, R)$ and with $||\hat{g}(x_n) - \nabla f(x_n)|| \leq \epsilon$ then

$$||x_n - x^*|| \leq \zeta^n ||x_0 - x^*|| + \frac{\eta\epsilon(1-\zeta^n)}{1-\zeta}$$

Proof. 1) See Lemma 3.6. in Hardt et al. (2016). The projection is a non-expensive map, so the result does not change.
2) Let us define $G'(x) = \Gamma[x + \eta\hat{g}(x)]$. Let us also define $\delta_n := ||x_{n+1} - x^*||$ Note that

$$\begin{aligned}\delta_{n+1} = ||x_{n+1} - x^*|| &= ||G'(x_n) - G(x^*)|| \\ &\leq ||G'(x_n) - G(x_n)|| \\ &\quad + ||G(x_n) - G(x^*)|| \\ &\leq \eta\epsilon + \zeta\delta_n.\end{aligned}$$

implying that

$$\delta_{n+1} \leq \zeta^n \delta_0 + \frac{\eta\epsilon(1-\zeta^n)}{1-\zeta}.$$

□

To conclude the proof, we need the following assumption on our problem. The first assumption is about the shape of local maxima.
Assumption 1: There exists $\pi^* \in \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \leq \phi\}$ such that:

- π^* is local maximum of $J(\pi)$;
- It exists R such that $\forall \pi \in B(\pi^*, R) \cap \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \leq \phi\}$, $J(\pi)$ is L-smooth and C-strongly concave.

The second assumption is to ensure that we can use the implicit differentiation theorem:
Assumption 2: We define

$$g(\pi, h) := [(1-\pi_i) \sum_{j\in V} P_{ij} h_j]_{i\in V/T}.$$

For all $\forall \pi \in B(\pi^*, R) \cap \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \leq \phi\}$, $(I - \nabla_h g(\pi, h(\pi))$ is invertible, where $h(\pi)$ is the solution of $h(\pi) = g(\pi, h(\pi))$.
Assumption 3: Let us define $\hat{g}(\pi) = [\lambda_i^n \sum_j P_{ij} h_j^n]_{i\in V/T}$. We assume that K is chosen such that $||\hat{g}(\pi) - \nabla J(\pi)|| \leq \epsilon$ for all $\forall \pi \in B(\pi^*, R) \cap \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \leq \phi\}$.

Now we are ready to state our main result:

Theorem 3.4. *If assumptions 1, 2 and 3 are satisfied, then π^n produces by the message passing algorithm verifies:*

$$\begin{aligned}\max_\pi J(\pi) \geq J(\pi^n) \geq \frac{1}{2} \max_\pi J(\pi^n) \\ - \frac{L}{2}(\zeta^n D + \frac{\eta\epsilon(1-\zeta)}{(1-\zeta)})^2,\end{aligned}$$

with η such that $\zeta := 1 - \frac{\eta CL}{C+L} < 1$ and D is the diameter of $B(\pi^*, R) \cap \{||\pi - \ ||| \sum_{i\in V} \pi_i \leq \phi\}\}$.

Proof. First note that we have from Lemma 3.2, assumption 1 and theorem 4.2 of Hassani et al. (2017):

$$\max_\pi J(\pi) \geq J(\pi^*) \geq \frac{1}{2} \max_\pi J(\pi)$$

Moreover by using the fact that for all $\pi \in B(\pi^*, R) \cap \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \leq \phi\}$, $J(\pi)$ is L-smooth we have that:

$$J(\pi) \geq J(\pi^*) - \frac{L}{2}||\pi - \pi^*||^2$$

implying that

$$\max_\pi J(\pi) \geq J(\pi) \geq \frac{1}{2} \max_\pi J(\pi) - \frac{L}{2}||\pi - \pi^*||^2$$

In particular, if we take $\pi = \pi^n$, $\pi^0 \in B(\pi^*, R) \cap \{\pi \in [0,1]^{|V|} | \sum_{i\in V} \pi_i \geq \phi\}$ and by using assumption 3 and lemma 3 we get:

$$\begin{aligned}\max_\pi J(\pi) \geq J(\pi^n) &\geq \frac{1}{2} \max_\pi J(\pi^n) - \frac{L}{2}||\pi^n - \pi^*||^2 \\ &\geq \frac{1}{2} \max_\pi J(\pi^n) \\ &\quad - \frac{L}{2}(\zeta^n D + \frac{\eta\epsilon(1-\zeta^n)}{(1-\zeta)})^2,\end{aligned}$$

which concludes our proof. □

4. NEURALIZED GRADIENT ASCENT: RANDOM BASIS APPROACH

In the previous section, we showed that a message passing algorithm can be used to derive a provable approximation of the optimal solution of the Random Walk Interdiction problem (2)-(5). We now use this first result to derive random basis that can be used to learn the optimal solutions. A natural way to design a random basis and to have good approximation guarantees is to create a basis from (17)-(19). For a given (π, P), we define the one-step map:

$$\left[\mathcal{T}_{\pi,P}(h,\lambda)\right]_i := \begin{cases} \Big[(1-\pi_i)\sum_j P_{ij} h_j, \\ -w_i + \sum_{j\neq T} \lambda_j (1-\pi_j) P_{ji}\Big], & i \neq T, \\ [1,0], & i = T, \end{cases}$$

and generate the simple fixed-point iterates by

$$\begin{aligned}&[h^{(k+1)}(\pi_\theta, P), \lambda^{(k+1)}(\pi_\theta, P)] \\ &= \mathcal{T}_{\pi_\theta,P}\big(h^{(k)}(\pi_\theta, P), \lambda^{(k)}(\pi_\theta, P)\big),\end{aligned}$$

with $[h^{(0)}, \lambda^{(0)}] \in \mathbb{R}^{2|V|}$, $[h_T^{(0)},\ \lambda_T^{(0)}] = [1,0]$. The learned approximation of the gradient is then a linear combination of these iterates,

$$[f(\pi_\theta, P, \theta)]_i := \sum_{k=0}^{K_\theta} \theta_k \left(\lambda_i^{(k)}(\pi_\theta, P) \sum_j P_{ij} h_j^{(k)}(\pi_\theta, P)\right).$$

Using this approximated gradient we can derive the following π_θ by simply replacing the true gradient by $f(\pi_\theta, P, \theta)$

in the gradient ascent algorithm (17)-(19). Such approach leads to the following three possible π_θ:

$$\pi_\theta = \Gamma_\Pi[\pi_\theta + \eta f(\pi_\theta, P, \theta)]\,, \qquad \text{(IP-NN)}$$

$$\pi_\theta^{(n+1)} = \Gamma_\Pi\Big[\pi_\theta^{(n)} + \eta f(\pi_\theta^{(n)}, P, \theta)\Big]\,, \qquad \text{(UN-NN)}$$

where IP-NN is a deep equilibrium network Bai et al. (2019), UN-NN is the unrolled-version of the IP-NN . Now we can construct a random basis for our problem as follows:

(1) Generate $\theta_1, \ldots, \theta_K$ drawn in a i.d.d. fashion from a distribution $G(\theta)$.
(2) Define the estimate $\pi_{\alpha,\theta} = \sum_{k=1}^{K_\alpha} \alpha_k \pi_\theta(P)$, with $\alpha \in \Delta(K_\alpha)$, where $\Delta(K_\alpha) := \{\alpha \in [0,1]^{K_\alpha} |\ \sum_{k=1}^{K_\alpha} \alpha_k = 1\}$ is simplex of dimension $K_\alpha - 1$.

Remark 4.1. *One the advantage of this approach is that we don't need to differentiate in θ and therefore we don't need to worry about the fact that the projection operator is non-differentiable which can lead to particular issues: 1) it is hard to understand theoretically where the algorithm is converging. Some attemps have been done using the notion of conservative gradient, but the current theory is only prove that a gradient subgradient is actually converging to a stationnary point Bolte and Pauwels (2021); Bolte et al. (2022). 2) Also it can lead to the zero gradient issue identified in Action-Constrained Reinforcement Learning Lin et al. (2021).*

Now that we have constructed our random basis, we would like to quantify its approximation quality. To this end, we show that there exists a coefficient vector α that guarantees a given performance level in expectation. This follows the standard approach introduced in Rahimi and Recht (2008). Let us first assume that:

Assumption 4:

(4.1) *Uniform boundedness:* $\sup_{\theta\in\Theta} \sup_{P\in\mathcal{P}} \|\pi_\theta(P)\|_2 \le B < \infty$.
(4.2) *Global Lipschitz regularity:* There exists $L_\theta > 0$ such that for all $\theta, \theta' \in \Theta$,
$$\sup_{P\in\mathcal{P}} \|\pi_\theta(P) - \pi_{\theta'}(P)\|_2 \le L_\theta \|\theta - \theta'\|_2.$$

We define the kernel $\alpha_\Delta(u) = \Delta^{-d} W(u/\Delta)$, where $W(u) \ge 0$, $\int_{\mathbb{R}^d} W(u)\,du = 1$, and $\mu_1 := \int \|u\|_2\, W(u)\,du < \infty$. The shifted kernel $\alpha_\Delta(\theta - \mathbf{1})$ approximates the Dirac at $\mathbf{1}$ as $\Delta \to 0$. The α of interest is defined as follow: We draw $\theta_1, \ldots, \theta_K \overset{\text{i.i.d.}}{\sim} G$ with density g on Θ, and define

$$\alpha_\Delta(\theta) := \frac{\alpha_\Delta(\theta - \mathbf{1})}{g(\theta)},$$

We also have the following assumption:

Assumption 5: We assume that $M_{\alpha,\Delta} := \sup_{\theta\in\Theta} \alpha_\Delta(\theta) < \infty$. Moreover, we assume that for all $\theta \in \Theta$, and for all $P \in \mathcal{P}$ and for all Δ there exists a constant $B > 0$ such that $\|\alpha_\Delta(\theta)\pi_\theta(P)\|_2 \le B$.

We focus on the performance of the following estimator:

$$\widehat{\pi}_{K_\alpha,\Delta}(P) := \frac{1}{K_\alpha} \sum_{k=1}^{K_\alpha} \alpha_\Delta(\theta_k)\, \pi_{\theta_k}(P).$$

Proposition 4.2. *Let $\Theta \subset \mathbb{R}^d$ and $\pi(P)$ be a reference mapping (for instance, $\pi^*(P)$). If assumption 4 and 5 are satisfied, then for any $\varepsilon \in (0,1)$, with probability at least $1-\varepsilon$ over the random draw $\{\theta_k\}$,*

$$\begin{aligned}
&\sup_{P\in\mathcal{P}} \|\widehat{\pi}_{K_\alpha,\Delta}(P) - \pi(P)\|_2 \\
&\le \sup_P \|\pi_{\mathbf{1}}(P) - \pi(P)\|_2 \;+\; L_\theta\, \mu_1\, \Delta \\
&+ B\sqrt{\tfrac{2\log(2/\varepsilon)}{K_\alpha}}.
\end{aligned}$$

Proof. First we have to identify the different terms that needs to be bounded. By construction we have,

$$\begin{aligned}
\mathbb{E}\big[\widehat{\pi}_{K_\alpha,\Delta}(P)\big] &= \int \alpha_\Delta(\theta)\, \pi_\theta(P)\, g(\theta)\, d\theta \\
&= \int \alpha_\Delta(\theta - \mathbf{1})\, \pi_\theta(P)\, d\theta =: (\mathcal{W}_\Delta \pi_\cdot(P))(\mathbf{1}).
\end{aligned}$$

Hence, for any P,

$$\begin{aligned}
&\|\widehat{\pi}_{K_\alpha,\Delta}(P) - \pi(P)\|_2 \\
&\le \|\widehat{\pi}_{K_\alpha,\Delta}(P) - \mathbb{E}\widehat{\pi}_{K_\alpha,\Delta}(P)\|_2 + \|\mathbb{E}\widehat{\pi}_{K_\alpha,\Delta}(P) - \pi_{\mathbf{1}}(P)\|_2 \\
&+ \|\pi_{\mathbf{1}}(P) - \pi(P)\|_2.
\end{aligned}$$

First using assumption 4.2 with $\theta' = \mathbf{1}$ and the change of variables $u = (\theta - \mathbf{1})/\Delta$,

$$\begin{aligned}
&\|\mathbb{E}\widehat{\pi}_{K_\alpha,\Delta}(P) - \pi_{\mathbf{1}}(P)\|_2 \\
&= \Big\| \int \alpha_\Delta(\theta - \mathbf{1})\big(\pi_\theta(P) - \pi_{\mathbf{1}}(P)\big)\, d\theta \Big\|_2 \\
&\le \int \alpha_\Delta(\theta - \mathbf{1})\, L_\theta\, \|\theta - \mathbf{1}\|_2\, d\theta \\
&= L_\theta\, \Delta \int \|u\|_2 W(u)\, du = L_\theta\, \mu_1\, \Delta.
\end{aligned}$$

Step 3: Sampling deviation. Each summand satisfies $\|\alpha_\Delta(\theta_k)\pi_{\theta_k}(P)\|_2 \le B$ by assumption 5. By using a vector-valued Hoeffding inequality (lemma 4 of Rahimi and Recht (2008)), for any $\varepsilon \in (0,1)$, with probability at least $1-\varepsilon$,

$$\|\widehat{\pi}_{K_\alpha,\Delta}(P) - \mathbb{E}\widehat{\pi}_{K_\alpha,\Delta}(P)\|_2 \;\le\; B\sqrt{\frac{2\log(2/\varepsilon)}{K_\alpha}}.$$

This bound is uniform in P since the constants are independent of P. Which concludes the proof.

□

Remark 4.3. *The assumption 4 follow from the contraction property of $T_{A,\alpha}$. Moreover, the first term in (4.2), $\sup_P \|\pi_{\mathbf{1}}(P) - \pi(P)\|_2$, corresponds exactly to the error introduced by using an approximated gradient and therefore Lemma 3.3 can be used to bound to find a bound on $\|\pi_{\mathbf{1}}(P) - \pi(P)\|_2$.*

5. LEARNING ALGORITHM

In this section, we focus on deriving an efficient Monte-Carlo method to estimate the gradient when we learn α in $\pi_{\alpha,\theta}$. For each node $i \in \mathcal{V}$, recall from lemma 3.2 that

$$h_i = \sum_{p\in\mathcal{P}_{i\to T}} \left(\prod_{\ell=0}^{k_p-1} (1 - \pi_{i_\ell}) \prod_{\ell=0}^{k_p-1} P_{i_\ell i_{\ell+1}} \right), \qquad (20)$$

where $\mathcal{P}_{i\to T}$ denotes the set of all finite walks from i to the target node T and k_p is the length of walk $p = (i_0 =$

$i, i_1, \ldots, i_{k_p} = T$). When $\pi_i = 0$ for all i, the probability of sampling p in the corresponding Markov chain is

$$\mathbb{P}_P(p \mid i_0 = i) = \prod_{\ell=0}^{k_p-1} P_{i_\ell i_{\ell+1}}. \tag{21}$$

Hence, for any parameter vector α,

$$h_i(\alpha) = \mathbb{E}_{p\sim\mathbb{P}_P(\cdot|i_0=i)}\left[\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right]. \tag{22}$$

Since the sampling distribution of a walk does not depend on α, we can compute $\nabla_\alpha h_i(\alpha)$ by differentiating under the expectation. Applying the standard *score-function* trick (Rubinstein and Kroese, 2016), we obtain the following result.

Proposition 5.1 (Score-function gradient estimator)**.** *For any transition matrix P and parameter vector α, the gradient of $h_i(\alpha)$ defined in* (22) *is*

$$\nabla_\alpha h_i(\alpha) \tag{23}$$
$$= -\,\mathbb{E}_{p\sim\mathbb{P}_P(\cdot|i_0=i)}\left[\left(\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right)\sum_{\ell=0}^{k_p-1}\frac{\nabla_\alpha\pi_{i_\ell}(\alpha)}{1-\pi_{i_\ell}(\alpha)}\right].$$

Consequently, the single-path estimator

$$g(p;\alpha) = -\left(\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right)\sum_{\ell=0}^{k_p-1}\frac{\nabla_\alpha\pi_{i_\ell}(\alpha)}{1-\pi_{i_\ell}(\alpha)}$$

is an unbiased estimate of $\nabla_\alpha h_i(\alpha)$.

Proof. We simply need to focus on computing the gradient of $\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))$. Note that $\nabla_\alpha \prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))$ is equal to

$$\left(\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right)\nabla_\alpha \log\left(\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right).$$

Observe that

$$\nabla_\theta \log\left(\prod_{\ell=0}^{k_p-1}(1-\pi_{i_\ell}(\alpha))\right) = -\sum_{l=0}^{k_p-1}\frac{\nabla_\theta\pi_{i_\ell}(\alpha)}{1-\pi_{i_\ell}(\alpha)},$$

which allows us to conclude the proof. □

For a fixed transition matrix P, the gradient in (23) can be estimated via Monte-Carlo sampling of paths.

In Algorithm 2, we now extend the gradient estimation procedure of Algorithm 1 to the stochastic setting where the transition matrix P itself is drawn from a distribution $\mathcal{P}$. At each iteration, a random environment $P \sim \mathcal{P}$ and a starting node $i \sim \rho$ are selected, the unbiased gradient estimator of Proposition 5.1 is computed using Algorithm 1, and a projected stochastic-gradient step is performed on α.

Remarks. (i) Algorithm 1 provides an unbiased estimate of $\nabla_\alpha h_i(P,\alpha)$ for any fixed P. (ii) The stochastic iteration in Algorithm 2 thus yields an unbiased estimator of $\nabla_\alpha J(\alpha)$. (iii) Standard results on stochastic approximation ensure convergence under diminishing step sizes and bounded variance.

Algorithm 1 Monte-Carlo estimator of $\nabla_\alpha h_i(\alpha)$ for fixed P

Require: Transition matrix P, start node i, target node T, parameterized removal probabilities $\pi(\cdot;\alpha)$, number of samples M

Ensure: Estimate $\widehat{\nabla_\alpha h_i}$

1: $\widehat{g} \leftarrow 0$
2: **for** $m = 1$ to M **do**
3: **Sample a path:** start from $i_0 = i$, draw $i_{\ell+1} \sim P_{i_\ell\cdot}$ until $i_k = T$; store $p = (i_0, \ldots, i_k = T)$
4: **Compute:**
$$W(p;\alpha) = \prod_{\ell=0}^{k-1}(1-\pi_{i_\ell}(\alpha)), \quad S(p;\alpha) = \sum_{\ell=0}^{k-1}\frac{\nabla_\alpha\pi_{i_\ell}(\alpha)}{1-\pi_{i_\ell}(\alpha)}$$
5: $g(p;\alpha) \leftarrow -\,W(p;\alpha)\,S(p;\alpha)$
6: $\widehat{g} \leftarrow \widehat{g} + g(p;\alpha)$
7: **return** $\widehat{\nabla_\alpha h_i} \leftarrow \widehat{g}/M$

Algorithm 2 Stochastic Gradient Algorithm with Environment Sampling

Require: Distribution $\mathcal{P}$ over transition matrices; node sampling ρ on V; learning rate schedule $\{\eta_t\}$; projection operator $\Gamma_{\Delta(K)}$ on the simplex of dimension $K-1$; initial parameter α_0.

Ensure: Sequence $\{\alpha_t\}$ approaching a stationary point of $J(\alpha) = \mathbb{E}_{P\sim\mathcal{P}}[F(P,\alpha)]$.

1: **for** $t = 0, 1, 2, \ldots$ **do**
2: **Sample environment:** draw $P^{(t)} \sim \mathcal{P}$.
3: **Sample start node:** draw $i^{(t)} \sim \rho$.
4: **Estimate gradient:** call
$$\widehat{\nabla_\alpha h_{i^{(t)}}}(P^{(t)},\alpha_t) \;\leftarrow\; \text{Algorithm } 1\big(P^{(t)},\, i^{(t)},\, \alpha_t\big).$$
5: **Form global gradient estimate:**
$$\widehat{G}_t = w_{i^{(t)}}/\rho(i)\,\widehat{\nabla_\alpha h_{i^{(t)}}}(P^{(t)},\alpha_t),$$
6: **Update parameters:**
$$\alpha_{t+1} = \Gamma_{\Delta(K)}\Big(\alpha_t - \eta_t\,\widehat{G}_t\Big).$$

6. NUMERICAL STUDIES

We evaluate our algorithms on randomly generated directed acyclic graphs (DAGs). To generate these graphs, we specify the number of layers and the number of nodes per layer. Each node in layer l is then connected to nodes in the next layer $l+1$ with a fixed probability (in this paper, we fix it at 0.25). Once the graph is generated, we derive the corresponding transition matrix by assigning transition probabilities from a node i to each of its out-neighbors j proportionally to the out-degree of those neighbors.

We simulate DAGs (20 for training and 10 for testing) with 30 nodes, 5 layers, a budget ϕ of 4.0, and $w_i = \frac{1}{|V|}$. For all experiments, we fix the random-basis parameters to $K_\alpha = 8$ and $K_\theta = 4$ and the number of unrolled iterations to $n = 6$. The step size inside each unrolled update is set to be equal to 0.08. To train our algorithm, we first use momentum gradient on 100 iterations and with a learning rate of 0.6 that decays exponentially with factor 0.02. For this first set of experiments, we don't use the Algorithm 2. We compute the gradient in π using implicit differentiation

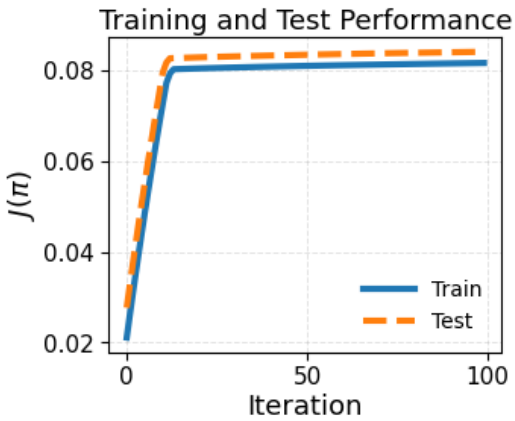

Fig. 1. Evolution of the training and testing curves.

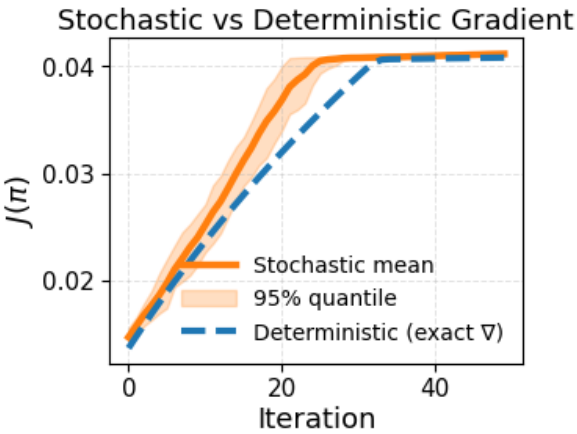

Fig. 2. Evolution of the stochastic gradient ascent vs. the determinist gradient ascent

and then we use the chain rule to get the gradient in α. The first observation is that when we look at figure 1, we see that we are actually able to train our neural networks to solve the optimisation problem even if n is small (number of gradient step in the random basis). We can also observe that the testing curve of our neural network (testing curve) is actually good compare to training curve. We believe that it is due to the fact the test set and the train set is generated on small random networks.

We now analyze Algorithm 2 and compare it with its non-stochastic counterpart. To capture the variability of the stochastic updates, we perform 10 independent runs of the stochastic gradient algorithm and display the empirical 95% quantile envelope across trajectories. As illustrated in Figure 2, the stochastic method converges to the same solution as the deterministic gradient descent, but it stabilizes faster, reaching steady performance around iteration 20, compared to approximately iteration 30 for the non-stochastic baseline.

7. CONCLUSION

In this paper, we introduced the *Random Walk Interdiction* (RWI) problem, a stochastic variant of the classical shortest-path interdiction problem in which an attacker follows a random walk toward a target. We showed that the optimal interdiction strategy can be approximated using a message-passing algorithm and quantify mathematically the approximation error. Building on this result, we proposed a random basis representation of the message passing algorithm and derived an efficient learning algorithm to optimize the associated weights (L2O approach). Our numerical experiments demonstrated the promises of this random basis approach. We can naturally think of different extensions for this work. Firstly, we could jointly optimize both the basis parameters θ and the combination coefficients α, to improve expressivity of our basis (which would be now a neural network) at the cost of higher computational complexity. Moreover, theoretically speaking, the study of the convergence of classical gradient algorithms in such case is known to be hard, and will require heavy mathematical tools (see Bach and Chizat (2021) for instance). Secondly, the current formulation assumes a fixed attacker dynamics (P is not changing). Extending the model to a setting where an adversary controls or adapts the transition matrix P would lead to a min–max formulation. Thirdly, the framework can be naturally extended to multiple-target settings by adding constraints similar to (3) for each transition matrix. This last extension does not require any change in the methodology and we believe that our approach can be directly applied in this case.

8. ACKNOWLEDGMENT

Alexandre Reiffers-Masson's research is supported by the Indo-French Centre for the Promotion of Advanced Research—CEFIPRA (7102-1).

9. DECLARATION OF GENERATIVE AI

During the preparation of this work the author(s) used Chatgpt in order to enhance the writing, editing, and drafting process. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

REFERENCES

Bach, F. (2019). Submodular functions: from discrete to continuous domains. *Mathematical Programming*, 175, 419–459.

Bach, F. and Chizat, L. (2021). Gradient descent on infinitely wide neural networks: Global convergence and generalization. *arXiv preprint arXiv:2110.08084*.

Bai, S., Kolter, J.Z., and Koltun, V. (2019). Deep equilibrium models. *Advances in neural information processing systems*, 32.

Bolte, J. and Pauwels, E. (2021). Conservative set valued fields, automatic differentiation, stochastic gradient methods and deep learning. *Mathematical Programming*, 188(1), 19–51.

Bolte, J., Pauwels, E., and Vaiter, S. (2022). Automatic differentiation of nonsmooth iterative algorithms. *Advances in Neural Information Processing Systems*, 35, 26404–26417.

Borkar, V.S. and Reiffers-Masson, A. (2021). Opinion shaping in social networks using reinforcement learning. *IEEE Transactions on Control of Network Systems*, 9(3), 1305–1316.

Chen, T., Chen, X., Chen, W., Heaton, H., Liu, J., Wang, Z., and Yin, W. (2022). Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23(189), 1–59.

Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing linear programs by graph neural networks. In *International Conference on Learning Representations*.

Guo, M., Li, J., Neumann, A., Neumann, F., and Nguyen, H. (2022). Practical fixed-parameter algorithms for defending active directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 9360–9367.

Hardt, M., Recht, B., and Singer, Y. (2016). Train faster, generalize better: Stability of stochastic gradient descent. In *International conference on machine learning*, 1225–1234. PMLR.

Hassani, H., Soltanolkotabi, M., and Karbasi, A. (2017). Gradient methods for submodular maximization. *Advances in Neural Information Processing Systems*, 30.

Lin, J.L., Hung, W., Yang, S.H., Hsieh, P.C., and Liu, X. (2021). Escaping from zero gradient: Revisiting action-constrained reinforcement learning via frank-wolfe policy optimization. In *Uncertainty in Artificial Intelligence*, 397–407. PMLR.

Ngo, H.Q., Guo, M., and Nguyen, H. (2024). Catch me if you can: Effective honeypot placement in dynamic ad attack graphs. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, 451–460. IEEE.

Qian, C., Chételat, D., and Morris, C. (2024). Exploring the power of graph neural networks in solving linear optimization problems. In *International conference on artificial intelligence and statistics*, 1432–1440. PMLR.

Rahimi, A. and Recht, B. (2008). Weighted sums of random kitchen sinks: Replacing minimization with randomization in learning. *Advances in neural information processing systems*, 21.

Reiffers-Masson, A., Shimkin, N., Menasche, D.S., and Altman, E. (2021). A backpropagation approach for distributed resource allocation.

Rubinstein, R.Y. and Kroese, D.P. (2016). *Simulation and the Monte Carlo method*. John Wiley & Sons.

Sato, R., Yamada, M., and Kashima, H. (2019). Approximation ratios of graph neural networks for combinatorial problems. *Advances in Neural Information Processing Systems*, 32.

Sinha, A., Fang, F., An, B., Kiekintveld, C., and Tambe, M. (2018). Stackelberg security games: Looking beyond a decade of success. IJCAI.

Wu, L., Cui, P., Pei, J., Zhao, L., and Guo, X. (2022). Graph neural networks: foundation, frontiers and applications. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 4840–4841.

Yau, M., Karalias, N., Lu, E., Xu, J., and Jegelka, S. (2024). Are graph neural networks optimal approximation algorithms? *Advances in Neural Information Processing Systems*, 37, 73124–73181.